CRUCIBLE

Autonomous Multi-Agent Code Review & Arbitration Studio

Course: Flexi-CA3 / Multi-Agent Systems | Submission: Project Synopsis

1. PROBLEM STATEMENT

Software developers?particularly students, bootcamp graduates, and junior engineers?routinely receive code feedback that is either too narrow (a linter catching only style issues) or too shallow (a single AI prompt giving generic critiques). A single AI model tasked with reviewing code faces a fundamental trade-off: it can assess logical correctness, or security vulnerabilities, or documentation quality, but attempting all three simultaneously spreads its attention budget so thin that each perspective becomes superficial. Furthermore, traditional linters operate in isolation, lacking contextual understanding, multi-turn critique, or autonomous remediation.

2. SOLUTION & MULTI-AGENT ARCHITECTURE

Crucible is an autonomous multi-agent code evaluation system where four specialized agents collaborate, disagree, and critique each other across a shared blackboard to produce a comprehensive, multi-perspective code review:

? Logic Reviewer: Evaluates algorithmic correctness, off-by-one errors, and control flow using AST analysis & Radon cyclomatic complexity.
? Security Auditor: Scans for injection vulnerabilities (CWE-89 SQLi), token leakage (CWE-798), and unsafe APIs using Bandit SAST and pattern detection.
? Style & Doc Agent: Analyzes idiomatic formatting (PEP 8, ESLint, gofmt) and documentation completeness (PEP 257, JSDoc, Go doc).
? Orchestrator: Built on LangGraph, dynamically routes reviews, detects cross-agent conflicts, arbitrates disagreements with written rationales, and synthesizes a final severity-ranked report.

3. TARGET USER & PROJECT SCOPE

Crucible incorporates autonomous fault-injection fallback (Category C2) and an autonomous Auto-Fix & Refactor engine that remediates detected flaws into secure, production-ready code.
Crucible is designed for students learning software engineering, bootcamp developers preparing for technical interviews, and development teams seeking automated, high-assurance multi-perspective code auditing. The system accepts source code in multiple languages (Python, JavaScript, Go), optional runtime context, and provides a real-time reactive blackboard timeline, audit event logs, automated security patch application, and comprehensive compliance artifacts.